图论 Graph/

> 约定：**除 Hungarian（1-indexed 片段）与 HLD（1-indexed，根默认 1）外，均 0-indexed**（点号 0..n-1）。 依赖 Template/ihxnan.hpp 的类型别名（ll、vi、pii 等）。 **冲突提醒：** MaxFlow.hpp 与 MaxAssignment.hpp 都在文件作用域定义了 constexpr int inf，同一个文件只能粘贴其一（或改名）。

最大流（Dinic） MaxFlow.hpp

- **用法**：MaxFlow<ll> mf(n)；addEdge(u, v, c) 加有向边（自动建反向边）；flow(s, t) 返回最大流；minCut() 求完流后返回每个点是否在 S 侧；edges() 取每条边流量明细。
- **复杂度**：O(n^2 m)，一般图远快于此；单位容量图 O(E√E)。
- **注意**：**0-indexed**；文件作用域有 constexpr int inf = 1E9，与 MaxAssignment.hpp 的 inf 冲突（同文件只能粘贴其一，或改名）。

```
constexpr int inf = 1E9; // 全局无穷大（与 MaxAssignment.hpp 的 inf 冲突，勿同时粘贴）
template <class T>
struct MaxFlow { // 最大流（Dinic）：0-indexed
    struct _Edge {
        int to;   // 出边指向的点
        T cap;    // 剩余容量
        _Edge(int to, T cap) : to(to), cap(cap) {}
    };

    int n;
    std::vector<_Edge> e;             // 边表，成对存储：i 与 i^1 互为反向边
    std::vector<std::vector<int>> g;  // 邻接表存边下标
    std::vector<int> cur, h;          // cur: 当前弧优化；h: BFS 分层

    MaxFlow() {}
    MaxFlow(int n) {
        init(n);
    }

    void init(int n) {
        this->n = n;
        e.clear();
        g.assign(n, {});
        cur.resize(n);
        h.resize(n);
    }

    // BFS 分层，标到 t 即可提前退出
    bool bfs(int s, int t) {
        h.assign(n, -1);
        std::queue<int> que;
        h[s] = 0;
        que.push(s);
        while (!que.empty()) {
            const int u = que.front();
            que.pop();
            for (int i : g[u]) {
                auto [v, c] = e[i];
                if (c > 0 && h[v] == -1) { // 有剩余容量且未分层
                    h[v] = h[u] + 1;
                    if (v == t) {
                        return true;
                    }
                    que.push(v);
                }
            }
        }
        return false;
    }

    // 多路增广：从 u 出发最多推 f 的流量
    T dfs(int u, int t, T f) {
        if (u == t) {
            return f;
```

```
        }
        auto r = f;
        for (int &i = cur[u]; i < int(g[u].size()); ++i) { // 当前弧优化
            const int j = g[u][i];
            auto [v, c] = e[j];
            if (c > 0 && h[v] == h[u] + 1) { // 只走相邻层
                auto a = dfs(v, t, std::min(r, c));
                e[j].cap -= a;     // 正向边扣容量
                e[j ^ 1].cap += a; // 反向边加容量
                r -= a;
                if (r == 0) {
                    return f;
                }
            }
        }
        return f - r;
    }

    // 加一条 u→v 容量 c 的有向边，同时自动建反向边（u←v 容量 0）
    void addEdge(int u, int v, T c)
    {
        g[u].push_back(e.size()); // u 记录正向边下标
        e.emplace_back(v, c);     // 正向边（偶数下标 i）
        g[v].push_back(e.size());
        e.emplace_back(u, 0);     // 反向边（下标 i^1）
    }

    T flow(int s, int t) // 求 s → t 最大流
    {
        T ans = 0;
        while (bfs(s, t)) {
            cur.assign(n, 0);
            ans += dfs(s, t, std::numeric_limits<T>::max());
        }
        return ans;
    }

    // 最小割划分：返回 c[i] = true 表示 i 在 S 侧
    std::vector<bool> minCut() {
        std::vector<bool> c(n);
        for (int i = 0; i < n; i++) {
            c[i] = (h[i] != -1); // 残量网络仍可达的即 S 侧
        }
        return c;
    }

    struct Edge {
        int from;
        int to;
        T cap;
        T flow;
    };

    // 取所有边的 {from, to, 容量, 流量}
    std::vector<Edge> edges() {
        std::vector<Edge> a;
        for (int i = 0; i < e.size(); i += 2) {
            Edge x;
            x.from = e[i + 1].to;
            x.to = e[i].to;
            x.cap = e[i].cap + e[i + 1].cap;
            x.flow = e[i + 1].cap;
            a.push_back(x);
        }
```

```
        return a;
    }
};
```

费用流（SPFA 朴素版，默认） MinCostFlow.hpp

- **用法**：MinCostFlow<ll> mcf(n)；add(u, v, cap, cost)；flow(s, t) 返回 {最大流, 最小费用}；flow(s, t, k) 限制流量上限 k；edges() 取边明细。
- **复杂度**：SPFA 增广，够用于多数题目。
- **注意**：**0-indexed**；可直接加**负费用边**（要求图中无负费用环）；模板参数 T 同时决定容量与费用类型。

```
template <class T> struct MinCostFlow // 最小费用最大流（SPFA 朴素版）：0-indexed
{
    struct _Edge
    {
        int to;   // 出边指向的点
        T cap;    // 剩余容量
        T cost;   // 单位费用（反向边为 -cost）

        _Edge(int _to, T _cap, T _cost) : to(_to), cap(_cap), cost(_cost)
        {
        }
    };

    struct Edge // 对外边明细
    {
        int from;
        int to;
        T cap;
        T cost;
        T flow;
    };

    int n;
    vector<_Edge> e;  // 边表，成对存储（i 与 i^1 互为反向边）
    vector<vector<int>> g;
    vector<T> dis;
    vector<int> pre;  // 增广路前驱边

    MinCostFlow()
    {
    }

    MinCostFlow(int n_) : n(n_), g(n_)
    {
    }

    void init(int n_)
    {
        n = n_;
        e.clear();
        g.assign(n, {});
    }

    void add(int u, int v, T cap, T cost) // 正向容量 cap、费用 cost；反向容量 0、费用 -cost
    {
        g[u].push_back(e.size());
        e.emplace_back(v, cap, cost);
        g[v].push_back(e.size());
        e.emplace_back(u, 0, -cost);
    }

    // SPFA 找费用最短增广路
    bool spfa(int s, int t)
    {
        dis.assign(n, numeric_limits<T>::max());
```

```
        pre.assign(n, -1);
        vector<bool> inq(n, false);
        queue<int> que;
        dis[s] = 0;
        que.push(s);
        inq[s] = true;
        while (!que.empty())
        {
            int u = que.front();
            que.pop();
            inq[u] = false;
            for (int i : g[u])
            {
                int v = e[i].to;
                T cap = e[i].cap;
                T cost = e[i].cost;
                if (cap > 0 && dis[v] > dis[u] + cost) // 松弛
                {
                    dis[v] = dis[u] + cost;
                    pre[v] = i;
                    if (!inq[v])
                    {
                        inq[v] = true;
                        que.push(v);
                    }
                }
            }
        }
        return dis[t] != numeric_limits<T>::max(); // t 可达
    }

    // k 为流量上限，返回 {流量, 费用}
    pair<T, T> flow(int s, int t, int k = iINF)
    {
        T flow = 0;
        T cost = 0;
        while (spfa(s, t) && flow < k)
        {
            T aug = numeric_limits<T>::max();      // 增广量 = 路径最小残量
            for (int i = t; i != s; i = e[pre[i] ^ 1].to)
                aug = min(aug, e[pre[i]].cap);
            for (int i = t; i != s; i = e[pre[i] ^ 1].to)
            {
                e[pre[i]].cap -= aug;   // 正向扣
                e[pre[i] ^ 1].cap += aug; // 反向加
            }
            flow += aug;
            cost += aug * dis[t]; // 路径费用 = dis[t]
        }
        return make_pair(flow, cost);
    }

    vector<Edge> edges() // 取所有边的 {from, to, 容量, 费用, 流量}
    {
        vector<Edge> a;
        for (int i = 0; i < e.size(); i += 2)
        {
            Edge x;
            x.from = e[i + 1].to;
            x.to = e[i].to;
            x.cap = e[i].cap + e[i + 1].cap;
            x.cost = e[i].cost;
            x.flow = e[i + 1].cap;
            a.push_back(x);
```

```
        }
        return a;
    }
};
```

费用流（Dijkstra + 势优化，被卡时间再换） MinCostFlowDijkstra.hpp

- **用法**：与 SPFA 版同名同接口：`add(u, v, cap, cost)`、`flow(s, t)` 返回 `{最大流, 最小费用}`。
- **复杂度**：每轮增广 $O(m \log n)$，总 $O(\text{flow} \cdot m \log n)$。
- **注意**：**0-indexed**；要求**初始边费用非负**（用Johnson 势 `h` 消除负权后跑 Dijkstra）。

```
template <class T>
struct MinCostFlow {
    struct _Edge {
        int to;
        T cap;
        T cost;
        _Edge(int to_, T cap_, T cost_) : to(to_), cap(cap_), cost(cost_) {}
    };
    int n;
    vector<_Edge> e;  // 成对存储，i 与 i^1 互为反向边
    vector<vector<int>> g;
    vector<T> h, dis; // h: 势（Johnson）；dis: 缩权后距离
    vector<int> pre;

    bool dijkstra(int s, int t) {
        dis.assign(n, numeric_limits<T>::max());
        pre.assign(n, -1);
        priority_queue<pair<T, int>, vector<pair<T, int>>, greater<pair<T, int>>> que;
        dis[s] = 0;
        que.emplace(0, s);
        while (!que.empty()) {
            T d = que.top().first;
            int u = que.top().second;
            que.pop();
            if (dis[u] != d) { // 过期堆元素
                continue;
            }
            for (int i : g[u]) {
                int v = e[i].to;
                T cap = e[i].cap;
                T cost = e[i].cost;
                // 缩权：cost + h[u] - h[v] ≥ 0
                if (cap > 0 && dis[v] > d + h[u] - h[v] + cost) {
                    dis[v] = d + h[u] - h[v] + cost;
                    pre[v] = i;
                    que.emplace(dis[v], v);
                }
            }
        }
        return dis[t] != numeric_limits<T>::max();
    }

    MinCostFlow() {}
    MinCostFlow(int n_) {
        init(n_);
    }
    void init(int n_) {
        n = n_;
        e.clear();
        g.assign(n, {});
    }

    void add(int u, int v, T cap, T cost) // 正向容量 cap，反向容量 0、费用 -cost
    {
        g[u].push_back(e.size());
        e.emplace_back(v, cap, cost);
```

```
        g[v].push_back(e.size());
        e.emplace_back(u, 0, -cost);
    }

    pair<T, T> flow(int s, int t) // 返回 {最大流, 最小费用}
    {
        T flow = 0;
        T cost = 0;
        h.assign(n, 0);
        while (dijkstra(s, t)) {
            for (int i = 0; i < n; ++i) {
                h[i] += dis[i]; // 更新势: h[i] += 真实距离
            }
            T aug = numeric_limits<int>::max(); // 增广量 (注意此处按 int 上限取)
            for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
                aug = min(aug, e[pre[i]].cap);
            }
            for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
                e[pre[i]].cap -= aug;
                e[pre[i] ^ 1].cap += aug;
            }
            flow += aug;
            cost += aug * h[t]; // h[t] 此时即 s→t 真实最短费用
        }
        return make_pair(flow, cost);
    }

    struct Edge {
        int from;
        int to;
        T cap;
        T cost;
        T flow;
    };

    vector<Edge> edges() // 边明细
    {
        vector<Edge> a;
        for (int i = 0; i < e.size(); i += 2) {
            Edge x;
            x.from = e[i + 1].to;
            x.to = e[i].to;
            x.cap = e[i].cap + e[i + 1].cap;
            x.cost = e[i].cost;
            x.flow = e[i + 1].cap;
            a.push_back(x);
        }
        return a;
    }
};
```

强连通分量（Tarjan） SCC.hpp

- **用法**：SCC scc(n)； add(u, v) 有向边；work() 返回 bel，bel[i] 为 i 所在 SCC 编号。
- **复杂度**：O(n + m)。
- **注意**：**0-indexed**；编号为**逆拓扑序**（对边 u->v 有 bel[v] <= bel[u]）；递归实现，深图注意栈空间。

```
struct SCC { // 强连通分量（Tarjan）：0-indexed
    int n;
    vector<vector<int>> adj;   // 邻接表
    vector<int> stk;           // Tarjan 栈（存当前 SCC 候选点）
    vector<int> dfn, low, bel; // dfn: 时间戳；low: 追溯值；bel: 所属 SCC 编号
    int cur, cnt;              // cur: 时间戳计数；cnt: SCC 计数

    SCC() {}
    SCC(int n) {
        init(n);
    }

    void init(int n) {
        this->n = n;
        adj.assign(n, {});
        dfn.assign(n, -1);
        low.resize(n);
        bel.assign(n, -1);
        stk.clear();
        cur = cnt = 0;
    }

    void add(int u, int v) { // 有向边
        adj[u].push_back(v);
    }

    void dfs(int x) {
        dfn[x] = low[x] = cur++; // 时间戳
        stk.push_back(x);

        for (auto y : adj[x]) {
            if (dfn[y] == -1) { // 树边
                dfs(y);
                low[x] = min(low[x], low[y]);
            } else if (bel[y] == -1) { // 返祖边（y 还在栈中）
                low[x] = min(low[x], dfn[y]);
            }
        }

        if (dfn[x] == low[x]) { // x 是 SCC 根，弹出栈内成一分量
            int y;
            do {
                y = stk.back();
                bel[y] = cnt;
                stk.pop_back();
            } while (y != x);
            cnt++;
        }
    }

    vector<int> work() // 遍历所有点，返回 bel
    {
        for (int i = 0; i < n; i++) {
            if (dfn[i] == -1) {
                dfs(i);
            }
```

```
        }
        return bel;
    }
};
```

边双连通分量（EBCC）`EBCC.hpp`

- **用法**：`EBCC ebcc(n)`；`add(u, v)` 无向边；`work()` 返回 `bel`（每个点的边双编号）；`compress()` 得到缩点后的信息。
- **复杂度**：`O(n + m log m)`（`set` 去重）。
- **注意**：**0-indexed**；`work()` 只从点 `0` 出发一次 DFS，**要求图连通**（不连通需对每个未访问点分别调用）；文件作用域有全局 `set<pii> E`（记录遍历中遇到的边，用于处理重边）。

```
set<pii> E; // 去重边集合（处理重边）

struct EBCC { // 边双连通分量：0-indexed
    int n;
    vector<vector<int>> adj;   // 邻接表
    vector<int> stk;           // Tarjan 栈
    vector<int> dfn, low, bel; // dfn: 时间戳；low: 追溯值；bel: 所属边双编号
    int cur, cnt;              // cur: 时间戳计数；cnt: 边双计数

    EBCC() {}
    EBCC(int n) {
        init(n);
    }

    void init(int n) {
        this->n = n;
        adj.assign(n, {});
        dfn.assign(n, -1);
        low.resize(n);
        bel.assign(n, -1);
        stk.clear();
        cur = cnt = 0;
    }

    void add(int u, int v) { // 无向边
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void dfs(int x, int p) {
        dfn[x] = low[x] = cur++;
        stk.push_back(x);

        for (auto y : adj[x]) {
            if (y == p) { // 只跳过一条父边，重边得以保留
                continue;
            }
            if (dfn[y] == -1) { // 树边
                E.emplace(x, y);
                dfs(y, x);
                low[x] = min(low[x], low[y]);
            } else if (bel[y] == -1 && dfn[y] < dfn[x]) { // 返祖边
                E.emplace(x, y);
                low[x] = min(low[x], dfn[y]);
            }
        }

        if (dfn[x] == low[x]) { // x 是边双根，弹出栈内成一个边双
            int y;
            do {
                y = stk.back();
                bel[y] = cnt;
                stk.pop_back();
            } while (y != x);
            cnt++;
```

```
        }
    }

    vector<int> work() // 返回 bel（从点 0 出发，图需连通）
    {
        dfs(0, -1);
        return bel;
    }

    // 缩点结果：n = 边双个数；edges 为边双间树边；siz 为各边双点数；cnte 为各边双内部边数
    struct Graph {
        int n;
        vector<pii> edges;
        vector<int> siz;
        vector<int> cnte;
    };
    Graph compress() {
        Graph g;
        g.n = cnt;
        g.siz.resize(cnt);
        g.cnte.resize(cnt);
        for (int i = 0; i < n; i++) {
            g.siz[bel[i]]++;
            for (auto j : adj[i]) {
                if (bel[i] < bel[j]) { // 跨边双：树边（只记一次方向）
                    g.edges.emplace_back(bel[i], bel[j]);
                } else if (i < j) { // 同边双内部边（无序只计一次）
                    g.cnte[bel[i]]++;
                }
            }
        }
        return g;
    }
};
```

圆方树（点双 → 块割树） BlockCutTree.hpp

- **用法**：`BlockCutTree bct(n)`；`add(u, v)` 无向边；`work()` 返回 `{方点个数, 树边集}`。
- **建树**：原点 `0..n-1`；每个点双（V-BCC）新建一个方点 `n+i`；返回的 `edges` 是方点与其中所有原点的连边。
- **复杂度**：`O(n + m)`。
- **注意**：**0-indexed**；递归实现。

```
struct BlockCutTree { // 圆方树（点双）：0-indexed，原点 0..n-1，方点 n+i
    int n;
    vector<vector<int>> adj;   // 原图邻接表
    vector<int> dfn, low, stk; // dfn: 时间戳；low: 追溯值；stk: 点栈
    int cnt, cur;              // cnt: 点双个数（方点编号）；cur: 时间戳计数
    vector<pii> edges;         // 圆方树的边（方点–原点）

    BlockCutTree() {}
    BlockCutTree(int n) {
        init(n);
    }

    void init(int n) {
        this->n = n;
        adj.assign(n, {});
        dfn.assign(n, -1);
        low.resize(n);
        stk.clear();
        cnt = cur = 0;
        edges.clear();
    }

    void add(int u, int v) { // 无向边
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void dfs(int x) {
        stk.push_back(x);
        dfn[x] = low[x] = cur++;

        for (auto y : adj[x]) {
            if (dfn[y] == -1) {
                dfs(y);
                low[x] = min(low[x], low[y]);
                if (low[y] == dfn[x]) { // y 的子树无法绕过 x：弹出一个点双
                    int v;
                    do {
                        v = stk.back();
                        stk.pop_back();
                        edges.emplace_back(n + cnt, v); // 方点连原点
                    } while (v != y);
                    edges.emplace_back(x, n + cnt); // 割点 x 也属于该点双
                    cnt++;
                }
            } else {
                low[x] = min(low[x], dfn[y]);
            }
        }
    }

    pair<int, vector<pii>> work() // 返回 {方点个数, 树边集}
    {
        for (int i = 0; i < n; i++) {
            if (dfn[i] == -1) {
```

```
                stk.clear();
                dfs(i);
            }
        }
        return {cnt, edges};
    }
};
```

2-SAT `TwoSat.hpp`

- **用法**：`TwoSat ts(n)`（变量 `0..n-1`）；`addClause(u, f, v, g)` 加约束 `(u==f) || (v==g)`（`f` / `g` 为期望布尔值）；`satisfiable()` 判可解；`answer()` 取方案。
- **复杂度**：`O(n + m)`。
- **注意**：变量 **0-indexed**；内部节点 `2i` 表示 `i` 取 `false`、`2i+1` 表示取 `true`；Tarjan 递归实现，规模大时注意栈空间。

```
struct TwoSat { // 2-SAT: 变量 0..n-1
    int n;                  // 变量个数
    vector<vector<int>> e;  // 蕴含图，节点 2i 表示 i=false、2i+1 表示 i=true
    vector<bool> ans;       // 每个变量的取值
    TwoSat(int n) : n(n), e(2 * n), ans(n) {}

    // 约束：(u=f) || (v=g)，即 !f→g、!g→f 两条蕴含边
    void addClause(int u, bool f, int v, bool g) {
        e[2 * u + !f].push_back(2 * v + g);
        e[2 * v + !g].push_back(2 * u + f);
    }

    bool satisfiable() {
        vector<int> id(2 * n, -1), dfn(2 * n, -1), low(2 * n, -1);
        vector<int> stk;
        int now = 0, cnt = 0;
        // 内嵌 Tarjan 求 SCC
        function<void(int)> tarjan = [&](int u) {
            stk.push_back(u);
            dfn[u] = low[u] = now++;
            for (auto v : e[u]) {
                if (dfn[v] == -1) {
                    tarjan(v);
                    low[u] = min(low[u], low[v]);
                } else if (id[v] == -1) {
                    low[u] = min(low[u], dfn[v]);
                }
            }
            if (dfn[u] == low[u]) {
                int v;
                do {
                    v = stk.back();
                    stk.pop_back();
                    id[v] = cnt;
                } while (v != u);
                ++cnt;
            }
        };
        for (int i = 0; i < 2 * n; ++i)
            if (dfn[i] == -1)
                tarjan(i);
        for (int i = 0; i < n; ++i) {
            if (id[2 * i] == id[2 * i + 1]) return false; // 同真同假：无解
            // Tarjan 逆拓扑序：编号大的分量先被选
            ans[i] = id[2 * i] > id[2 * i + 1];
        }
        return true;
    }

    vector<bool> answer() { return ans; }
};
```

二分图最大匹配（匈牙利） Hungarian.hpp

- **用法**：**代码片段**，整体粘贴进 solve() （[&] 捕获局部变量）。要求已有：左部到右部的邻接表 adj （**1-indexed**），全局 n 。
- **调用**：取消注释的循环，对左部点 1..m 逐个增广， cnt 即最大匹配数。
- **复杂度**： O(n · m) 。
- **注意**： match[p] 为右部点 p 当前匹配的左部点（ 0 表示未匹配，因此点号必须从 1 开始）。

```
vector<bool> visit(n + 1);
vector<int> match(n + 1);
auto dfs = [&](auto &&self, int x) → bool { // x: 左部点
    for (auto &p : adj[x])                  // p: 右部点
        if (!visit[p])
        {
            visit[p] = true;
            if (!match[p] || self(self, match[p])) // p 空闲，或其原匹配可换
            {
                match[p] = x;
                return true;
            }
        }
    return false;
};

// int cnt = 0;
// for (int i = 1; i ⩽ m; ++i)       // 枚举左部点增广
// {
//     fill(visit.begin(), visit.end(), false);
//     cnt += dfs(dfs, i);
// }
```

二分图最大权完美匹配（KM） MaxAssignment.hpp

- **用法**：MaxAssignment<ll> km; ll ans = km.solve(nx, ny, a); —— a 为 nx × ny 的非负权矩阵，要求 nx <= ny； assignment() 返回每个左部点匹配的右部点（xy[i]）； weights() 返回"恰好匹配前 k 个"的最大权前缀和数组。
- **复杂度**：O(n^2 m)。
- **注意**：**0-indexed**；文件作用域有 constexpr int inf = 1E7，与 MaxFlow.hpp 的 inf 冲突；权值须非负。

```
constexpr int inf = 1E7;
template <class T>
struct MaxAssignment {
    public:
        // 返回 nx 个左部点全部匹配的最大权值和
        T solve(int nx, int ny, vector<vector<T>> a) {
            assert(0 ⩽ nx && nx ⩽ ny);
            assert(int(a.size()) == nx);
            for (int i = 0; i < nx; ++i) {
                assert(int(a[i].size()) == ny);
                for (auto x : a[i])
                    assert(x ⩾ 0); // 权值非负
            }

            // 用 x 更新右部点的 slack（松弛量）与来源 slackx
            auto update = [&](int x) {
                for (int y = 0; y < ny; ++y) {
                    if (lx[x] + ly[y] - a[x][y] < slack[y]) {
                        slack[y] = lx[x] + ly[y] - a[x][y];
                        slackx[y] = x;
                    }
                }
            };

            costs.resize(nx + 1);
            costs[0] = 0;
            lx.assign(nx, numeric_limits<T>::max()); // 左部顶标
            ly.assign(ny, 0);                        // 右部顶标
            xy.assign(nx, -1);                       // 左 → 右匹配
            yx.assign(ny, -1);                       // 右 → 左匹配
            slackx.resize(ny);
            for (int cur = 0; cur < nx; ++cur) { // 逐个加入左部点
                queue<int> que;
                visx.assign(nx, false);
                visy.assign(ny, false);
                slack.assign(ny, numeric_limits<T>::max());
                p.assign(nx, -1); // 交错树上 x 的前驱

                for (int x = 0; x < nx; ++x) {
                    if (xy[x] == -1) { // 未匹配的 x 作为根
                        que.push(x);
                        visx[x] = true;
                        update(x);
                    }
                }

                int ex, ey;        // 找到的增广终点
                bool found = false;
                while (!found) {
                    // 1) 在相等子图内 BFS 找增广路
                    while (!que.empty() && !found) {
                        auto x = que.front();
                        que.pop();
                        for (int y = 0; y < ny; ++y) {
                            if (a[x][y] == lx[x] + ly[y] && !visy[y]) { // 相等边
```

```
                        if (yx[y] == -1) { // y 未匹配：增广成功
                            ex = x;
                            ey = y;
                            found = true;
                            break;
                        }
                        que.push(yx[y]); // 顺着匹配边走
                        p[yx[y]] = x;
                        visy[y] = visx[yx[y]] = true;
                        update(yx[y]);
                    }
                }
            }
            if (found)
                break;

            // 2) 相等子图内无增广路：调整顶标
            T delta = numeric_limits<T>::max();
            for (int y = 0; y < ny; ++y)
                if (!visy[y])
                    delta = min(delta, slack[y]);
            for (int x = 0; x < nx; ++x)
                if (visx[x])
                    lx[x] -= delta;
            for (int y = 0; y < ny; ++y) {
                if (visy[y]) {
                    ly[y] += delta;
                } else {
                    slack[y] -= delta;
                }
            }
            // 3) slack 减到 0 的边进入相等子图，继续增广
            for (int y = 0; y < ny; ++y) {
                if (!visy[y] && slack[y] == 0) {
                    if (yx[y] == -1) {
                        ex = slackx[y];
                        ey = y;
                        found = true;
                        break;
                    }
                    que.push(yx[y]);
                    p[yx[y]] = slackx[y];
                    visy[y] = visx[yx[y]] = true;
                    update(yx[y]);
                }
            }
        }

        // 沿交错路翻转匹配，顺便维护 costs 前缀
        costs[cur + 1] = costs[cur];
        for (int x = ex, y = ey, ty; x != -1; x = p[x], y = ty) {
            costs[cur + 1] += a[x][y];
            if (xy[x] != -1)
                costs[cur + 1] -= a[x][xy[x]]; // 减去旧匹配
            ty = xy[x];
            xy[x] = y;
            yx[y] = x;
        }
    }
    return costs[nx];
}
vector<int> assignment() { // 每个左部点的匹配对象
    return xy;
}
```

```
        pair<vector<T>, vector<T>> labels() { // 顶标
            return make_pair(lx, ly);
        }
        vector<T> weights() { // costs[k]: 只匹配前 k 个的最优权值和
            return costs;
        }
    private:
        vector<T> lx, ly, slack, costs;
        vector<int> xy, yx, p, slackx;
        vector<bool> visx, visy;
};
```

重链剖分（HLD，1-indexed）`HLD.hpp`

- **用法**：`HLD hld(n)`；`add(u, v)` 建无向树（点号 **1..n**）；`work(root=1)` 预处理；`lca(u, v)`、`dist(u, v)`、`jump(u, k)`（`u` 向上 `k` 步，不足返回 `0`）、`isAncester(u, v)`（`u` 是否为 `v` 祖先）、`rootedParent(u, v)`（以 `u` 为根时 `v` 的父亲）、`rootedSize(u, v)`（以 `u` 为根时 `v` 子树大小）、`rootedLca(a, b, c)`（三点 LCA，用异或消去）。
- **DFS 序**：`in[u]` 进栈时间戳、`out[u]` 出栈时间戳（子树对应区间 `[in[u], out[u])`）；`seq[t]` 时间戳对应的点；重儿子排在 `adj[u][0]`，**重链在 `in` 上连续**，可配合线段树维护链信息。
- **复杂度**：预处理 `O(n)`，LCA / jump 单次 `O(log n)`。

```
struct HLD { // 重链剖分：1-indexed，根默认 1
    int n, cur;                                     // cur：DFS 序时间戳（从 1 开始）
    vector<int> sz, top, dep, fa, in, out, seq; // 子树大小 / 链头 / 深度 / 父亲 / 进栈 / 出栈 / 时间戳对应点
    vector<vector<int>> adj;                    // 邻接表（重儿子会被换到 adj[u][0]）

    HLD(int n) : n(n), cur(1), sz(n + 1), top(n + 1), dep(n + 1), fa(n + 1), in(n + 1), out(n + 1), seq(n + 1), adj(n + 1) {}

    void add(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void work(int root = 1) { // 预处理（默认根为 1）
        top[root] = root;
        dep[root] = 0;
        fa[root] = 0;
        dfs1(root);
        dfs2(root);
    }

    // 第一遍：求 fa、dep、sz，并把重儿子换到 adj[u][0]
    void dfs1(int u) {
        if (fa[u] != 0) {
            adj[u].erase(find(adj[u].begin(), adj[u].end(), fa[u])); // 删掉父边
        }
        sz[u] = 1;
        for (auto &v : adj[u]) {
            fa[v] = u;
            dep[v] = dep[u] + 1;
            dfs1(v);
            sz[u] += sz[v];
            if (sz[v] > sz[adj[u][0]]) { // 重儿子放最前
                swap(v, adj[u][0]);
            }
        }
    }

    // 第二遍：求 DFS 序 in/out/seq 与 top（重链头）
    void dfs2(int u) {
        in[u] = cur++;
        seq[in[u]] = u;
        for (auto v : adj[u]) {
            top[v] = v == adj[u][0] ? top[u] : v; // 重儿子继承链头
            dfs2(v);
        }
        out[u] = cur;
    }

    int lca(int u, int v) {
        while (top[u] != top[v]) { // 不同链：链头深的先跳
```

```
            if (dep[top[u]] < dep[top[v]])
                swap(u, v);
            u = fa[top[u]];
        }
        if (dep[u] > dep[v]) // 同链：浅的为 LCA
            swap(u, v);
        return u;
    }

    int dist(int u, int v) { // 树上距离
        return dep[u] + dep[v] - 2 * dep[lca(u, v)];
    }

    int jump(int u, int k) { // u 向上跳 k 步；不足时返回 0
        if (dep[u] < k) {
            return 0;
        }

        int d = dep[u] - k; // 目标深度

        while (dep[top[u]] > d) { // 整链上跳
            u = fa[top[u]];
        }

        return seq[in[u] - dep[u] + d]; // 链内按 DFS 序直达
    }

    bool isAncester(int u, int v) { // u 是否为 v 的祖先（含自身）
        return in[u] ≤ in[v] && in[v] < out[u];
    }

    int rootedParent(int u, int v) { // 以 u 为根时 v 的父亲
        swap(u, v);
        if (u == v) {
            return u;
        }
        if (!isAncester(u, v)) {
            return fa[u];
        }
        // v 是根：找到 u 子树中 DFS 序最小的儿子方向
        auto it = upper_bound(adj[u].begin(), adj[u].end(), v, [&](int x, int y) {
            return in[x] < in[y];
        }) - 1;
        return *it;
    }

    int rootedSize(int u, int v) { // 以 u 为根时 v 的子树大小
        if (u == v) {
            return n;
        }
        if (!isAncester(v, u)) {
            return sz[v];
        }
        return n - sz[rootedParent(u, v)];
    }

    int rootedLca(int a, int b, int c) { // 三点 LCA：lca 两两异或
        return lca(a, b) ^ lca(b, c) ^ lca(c, a);
    }
};
```